Small Group Project Self-Assessment

Team name: Flax-6
URL of the deployed application: https://hamed54.pythonanywhere.com/
Team members:

- Arvin Hadidi-Fard
- Cate Tay Chew
- Hamed Latif
- Katherine Wanting Zhao
- Kausar Khan

If the team agrees on an effective team size, specify it here. Otherwise, state that the team does not agree.

The team agrees that the effective team size was five.

Submission checklist

Check	Task
	The work that is submitted has been produced by members of the team only, without input from other students or third parties. All use of examples from the internet and generative AI are declared in the `README.md` file.
	The team is familiar with the required deliverables for this project, outlined in the small group project handbook, available on KEATS.
	The team's shared GitHub repository has been registered on Team Feedback. All commits are attributed to a member of the team (check this on Team Feedback!). All collaborative coding sessions have been registered by the committer on Team Feedback. The shared repository has a default branch that corresponds to the accepted version of the software.
	The version of the web application that the team wishes to submit has been deployed. The URL is recorded in the `README.md` file. The deployed system is seeded with data that meets the requirements outlined in the specifications. The system is accessible using the required access credentials. The deployed system has been manually tested to ensure all functionality available in development works and is accessible in production.
	All team members listed above agree with the claims made in this checklist.

Objective

Briefly outline the business objective of the system you have built. Aim to identify what the system you developed aims to achieve (do not regurgitate the overall assignment). This section should consist of a single paragraph. Aim for less than 100 words.

This project aims to deliver a functional recipe management platform for students in shared accommodation who struggle with scattered recipe sources and meal planning. Our system integrates three core workflows: users can create and discover recipes with tags, plan weekly meals with a reusable calendar interface, and generate categorised shopping lists automatically from their meal plans. We incorporated AI-powered recipe generation to reduce decision fatigue and added social features, allowing users to follow others and discover popular recipes. The platform combines practical cooking with community features, making consistent home cooking more achievable for students managing budgets and busy schedules.

Use cases

Use cases clarify who uses the system and how they can use it to accomplish particular tasks. Identify the main use cases of your team's application. For each use case, provide:

- Name: a name for the use case. If the name does not describe the task adequately by itself, add a short task description.
- Actor & trigger: Identify who and what starts the interaction with the system.
- Main success scenario & outcome: List the main steps that lead to a successful outcome.
- Alternate flows: Identify possible alternative flow conditions (e.g. what happens under anomalous conditions).

An example of a use case is as follows:

- Name: Place order
- Actor and trigger: Customer selects products and proceeds to checkout.
- Main success scenario: (1) Customer logs in, (2) adds products to cart, (3) proceeds to checkout, (4) enters shipping and payment details, (5) confirms order, (6) system processes payment and displays confirmation. Outcome: new order is placed.
- Alternate flows: Payment fails → system redirects to step (4) with error message.

Name (& short task description)	**Actor & trigger**	**Main success scenario & outcome**	**Alternative flows**
Create Recipe	Logged-in user selects "Create Recipe" from the navigation bar	(1) System displays Step 1 form, (2) User enters title, publication date, servings, ingredients, steps, optional image, visibility and times. (3) User clicks "Next: Add Tags", (4) System displays Step 2 tag page, (5) User selects tags or adds custom tags, (6) User clicks "Create Recipe". Outcome: recipe is saved and shown on the detail page with a success message.	Missing fields → system shows validation errors. Invalid date → user must enter today or earlier. Image upload fails user may retry or continue without image. User cancels → no recipe is saved and user is taken back to main page.

Filter Recipe	Logged-in user opens the all recipes page by clicking "View All" on the dashboard page or by clicking "Discover → All Recipes" in the navigation bar.	(1) System displays all public recipes (default: newest first). (2) User selects one or more tag filters (Course, Dietary, Cuisine, Others). (3) User applies filters by pressing Return or clicking "Apply". Outcome: Active filters will be shown above all the recipes and below the filter buttons. (4) User clears filters to return to the full recipe list. Outcome: All recipes will be shown again (5) User can add to the filters by clicking (checking) more filters Outcome: Recipes shown gets further filtered to show recipes that contain all tag(s) selected	No alternate flows.
Sort Recipes	Logged-in user opens the all recipes page by clicking view all on the dashboard page or clicking Discover/All Recipes in the navigation bar.	(1) Users click the the drop down that is default "Newest First". (2) From the dropdown options (publication date, title, time, most saved, highest rated) the user will click what they want to sort the recipes by. Outcome: System displays recipes sorted based on the sort by option selected.	e.g. if "Prep Time (Least)" was selected, the recipes shown will start from the recipe with the shortest prep time. (3) If a filter selected before or after sorting, the filtered recipes will be sorted.

Update my own recipe	Logged-in User opens My Kitchen/My Recipes in the navigation bar to view all their own recipes. User clicks edit for the recipe they want to Update	(1) System displays the first page (Step 1) of the same form as the create recipe form, but with the fields already filled in with the existing recipe's details. (2) User can update any field they want by simply deleting and/or adding words/numbers. If there's an image present, it will be shown at the bottom of the page with a checkbox "Delete image". (3) Click the delete image checkbox to delete the image. (4) To replace the existing image, add a new image under the image field. (5) Click "Next: Add Tags" Outcome: System displays "Step 2" tags page. (6) In Step 2, the user can delete/add tags. (7) User clicks "Update Recipe" Outcome: System displays the recipe detail page for the updated Recipe. At the top of the page it will show that the recipe was successfully updated.	Missing fields → system shows validation errors. Invalid date → user must enter today or earlier. Image upload fails user may retry or continue without image. User cancels → recipe remains the same with no updates.

Create a modification of another user's recipe	Logged-in user views a recipe by clicking on it. If the recipe doesn't belong to them, they can create a modication by clicking "Create My Version" at the bottom of the page.	(1) System displays the "Step 1" of the create recipe form, but with the fields filled in with the existing recipe's details. At the top of the form, it will tell the user that it is creating a modified version of the original author's (showing the actual username) recipe. (2) User can add/delete to the fields. If there's an image present, it will be shown at the bottom of the page with a checkbox "Delete image". (3) Click the delete image checkbox to delete the image. (4) To replace the existing image, add a new image under the image field. (5) Click "Next: Add Tags" Outcome: System displays "Step 2" tags page. (6) In Step 2, the user can delete/add tags. (7) User clicks "Create My Version" Outcome: System displays the recipe detail page for the new Recipe created. At the top of the page it will show that the recipe was successfully created (authored by the user). The original recipe will remain untouched.	Missing fields → system shows validation errors. Invalid date → user must enter today or earlier. Image upload fails user may retry or continue without image. User cancels → original recipe remains unchanged.

Browse and Search Users	Logged-in user clicks "View All" in dashboard page or Discover/All Recipes in navigation bar. System displays the all recipes page. User clicks the user tab	(1) System displays all users (2) User clicks follow/unfollow. Outcome: System displays newly followed/unfollowed user. System tells the user at the top of the page that they followed/unfollowed the user shown. (3) In the view of all users, the user can view the profile by clicking the user card Outcome: System displays the profile of the user clicked (4) User can search by username (@johndoe), first name (john), last name (doe), or both (john doe) Outcome: System displays recipes made by users matching the search query.	User can click user tab to view users that match the search query. If no user/recipe matches the search query, no search results will be shown.
Search Recipes	Logged-in user clicks on the search bar in the navbar at the top of the home page. User enters search parameters. Relevant recipes are shown.	(1) User clicks on the searchbar (2) User enters search parameters in the searchbar Outcome: System displays all relevant recipes on the page, with a title saying 'Search results for SEARCH-PARAM'. (3) In the view of all recipes, the user can view any recipe by clicking the relevant recipe card Outcome: System displays the recipe.	No alternative workflow.
View Recipes	Logged-in user clicks on the 'View' button on one of the recipes, from the all-recipes page. System shows details about the recipe, on a new page.	(1) User clicks on the recipe. Outcome: System displays details about the recipe, such as the steps to make it, the ingredients needed, ratings, tags, time for preparation + cooking.	You can click 'View' on a recipe shown in other similar pages, such as the some recipes page.

Get Cost of Recipe	Logged-in user clicks on the 'Get Cost Estimate' button on a recipe. System displays a cost estimate about the recipe, on the same page.	(1) User clicks on the recipe. Outcome: System displays details about the recipe. (2) User clicks on the 'Get Cost Estimate' button. Outcome: System makes an AI call to get a cost estimate for the recipe, along with a per-portion estimate.	You could manually append "&show_cost=true" to the URL of the page.
Plan Weekly Meals	Logged-in user clicks on the 'Meal Plan' section of the dropdown menu after clicking the 'My Kitchen' button on the navbar. System displays a list of the 7 days of the week with options to add recipes for breakfast, lunch and dinner, on a new page.	(1) User clicks on the 'My Kitchen' button on the navbar. (2) User clicks on the 'Meal Plan' link on the dropdown menu. Outcome: System displays a list of the 7 days of the week with options to add recipes for breakfast, lunch and dinner, on a new page. There is also an option to add snacks for each day and generate a shopping list based on the meals for the week. (3) User can click on the 'Edit' button for any day, select any of the meals to edit. Outcome: System brings up a choice of all recipes for the user to insert. (4) User can choose any of the recipes in the menu. Outcome: The meal is added to the relevant part of the meal plan page, where the user is directed to.	No alternate workflow.

Generate Recipe with AI	Logged-in users can click on the 'Discover' button in the navbar and will then click on "Chef AI". The user writes a prompt to the AI describing what recipe they want and clicks the 'Submit' button labeled as an arrow right next to the prompt bar.	(1) User logs in and is directed to the dashboard (2) User clicks on the 'Discover' button in the nav bar. A drop-down menu with two options appears. (3) User clicks on the 'Chef AI' button in the drop-down menu. Outcome: System displays a user interface with a prompt bar, prompting the user to enter a description of what they want to eat. (4) User writes a prompt describing what they want to eat and press the submit button labeled as an arrow. Outcome: An API call is made to OpenAI model gpt-4o-mini, which answers the prompt. The page reloads and shows a UI with the recipe information. Including: The user prompt as a chat message; the title of the generated recipe; a short description of the recipe; Prep Time, Cook Time, Serving Size, Difficulty; Dietary Info (matched with tags from the existing tag system); ingredients; instructions; chef's tips. (5) User is able to generate another recipe by clicking on the "Generate Another Recipe" button. Outcome: The page reloads to a new, clear chat, prompting the user to describe what they want to eat.	(5) User can generate another recipe by either clicking on the button, or typing in another prompt in the prompt box. Outcome: The page reloads with a new, generated recipe based on the prompt submitted. (6) User can click the button "Save This Recipe". This will load and autofill in the recipe information into a Create Recipe form (Step 1), which the user will be able to edit and post.

Reporting Inappropriate Recipes	Logged-in user views a recipe and clicks on the Flag icon just above the Rating and just below the username of the recipe creator.	(1) User logs in and is directed to the dashboard. (2) User sees featured recipes in the dashboard, or under the "All recipes" section. (3) User clicks on "View" on a recipe card. They then may see some inappropriate content in the recipe card, e.g. wrong instructions. (4) User wants to report or flag this recipe for review. They click the Flag icon. Outcome: The recipe gets a Flag count of $+1$. An admin will be able to see flagged recipes and decide what to do.	A user cannot flag their own recipes.

Remove or Edit Users and Recipes	Admin logs in with Admin credentials and directs to the URL extension /admin. i.e. https://hamed54.pythonanywhere.com/admin/ Admin can see Flags, Ratings, Recipes, Users and can edit or delete correspinding data.	(1) Admin logs in with their credentials and is directed to the Dashboard. (2) Admin replaces /dashboard with /admin to the link, directing him to the admin panel. i.e. https://hamed54.pythonanywhere.com/dashboard -¿ https://hamed54.pythonanywhere.com/admin (3) Admin will be able to see 4 sections: Flags, Ratings, Recipes, Users. (4) Flags will show flagged recipes, including Recipe title, Recipe author, User (who flagged the recipe), Recipe rating, Flag count for recipe. (5) Ratings will show all rated recipes, including Recipe title, Recipe author, User (who rated), Recipe rating, Stars. (6) Recipes will show all information regarding a recipe. Clicking the name of the recipe will show all the meta tags associated, including the publication date, the steps, the image, tags, saved by, etc. (7) Users will show all information related to the user, including their name, email, age, gender, profile picture, followings. (8) By clicking on any of these fields, an admin has the ability to change whichever field they wanted to, and either "Save" changes, "Save and add another" change, "Save and continue editing", or "Delete", which will permanently delete the flag, rating, recipe or user.	- It's possible to go directly to https://hamed54.pythonanywhere.com/admin/ (or localhost/admin - depending on what version is tested) - to access the admin panel, where the user will be prompted to enter the admin log-in credentials. - An admin can directly delete recipes from the normal user interface. When clicking on "View" a recipe, by scrolling down all the way to the bottom, a button "Delete this recipe" can be triggered. When clicked by an admin, they will be prompted to confirm the deletion of the recipe. This change will then be reflected everywhere else in the app. The recipe will be permanently deleted from the database.